1 Calling conventions

In assembly, sometimes we need to make system calls to the OS. We need to follow the calling conventions to make sure that the OS can understand our request.

1.1 Start with examples

Start we an example:

```
    .text
    .global _start
_start:
    mov x0, 42
    mov x8, 93
    svc 0
```

Here, we are trying to make a system call to exit the program. We need to put the system call number in x8, and the argument in x0. The system call number for exit is 93. We then use svc 0 to make the system call.

- *.text* tells the assembler/linker that code follows.
- *.global* tells the assembler/linker that the symbol _start is visible to the linker. This is the entry point of the program.
- Then we put the value in x0 and x8. Here x0 is the argument for the system call, and x8 is the system call number.

To compile this and run (on Linux/arm64):

```
as -0 foo.o foo.s
ld -o foo foo.o
./foo
echo \$?
```

The last line prints the exit code of the program. In this case, it should be 42.

- *as* is the assembler. It takes the assembly code and produces an object file.

- *ld* is the linker. It takes the object file and produces an executable.
- *./foo* runs the executable.
- *echo*$? prints the exit code of the program.

Another example:

```
    .text
    .global _start
_start:
    mov x8, 64
    mov x0, 1
    adr x1, txt
    mov x2, len
    svc 0
    mov x8, 93
    mov x0, 42
    svc 0
txt: .ascii "goodbye, cruel world!\textbackslash n"
len = . - txt
```

This program prints "goodbye, cruel world!" and exits with code 42. Here, we are using the write system call. The system call number for write is 64. The arguments are:

- system call number 64 in x8 means write
- file descriptor 1 in x0 means stdout, which is the standard output.
- address of the string in x1
- length of the string in x2
- system call number 93 in x8 means exit
- exit code 42 in x0
- system call 0 to make the system call
- the string to print

- The length is calculated by subtracting the address of the string from the current location. (note that in the stack, the address grows downwards)

One more example:

```
length:
    mov x2, -1
length_rec:
    add x2, x2, 1
    ldrb w1, [x0], 1
    cbnz w1, length_rec
    mov x0, x2
    ret


.global _start
_start:
    adr x0, txt
    bl length
    mov x2, x0
    mov x0, 1
    adr x1, txt
    mov x8, 64
    svc 0
    mov x8, 93
    mov x0, 0
    svc 0
txt: .asciz "goodbye, cruel world!\textbackslash n"


    //Here we decompose bl and ret
    str lr, [sp, -4]! //store the lr original value
    mov lr, [pc, 4]
    //load the address of the next instruction
    mov pc, length //jump to the function
    ...\dots
    //now returning
    mov pc, lr //return to the address in lr
```

```
ldr lr, [sp] //restore the lr value
add sp, sp, 4 //restore the stack pointer
//that is it
```

Here instead of using the length variable, we are using a function to calculate the length of the string. The function is called length. It takes the address of the string as the argument and returns the length of the string in x0. The function is called using bl, which is a branch and link. It branches to the function and saves the address of the next instruction in the link register. The function returns using ret, which branches to the address in the link register.

Explain code line by line:

- $movx2, -1$ sets x2 to -1. This is the initial value of the length.
- $addx2, x2, 1$ increments x2 by 1. Increase the length by 1 for each loop.
- $ldrbw1, [x0], 1$ loads a byte from the address in x0 into w1. The address in x0 is then incremented by 1. This loads the next character in the string. If we want to increment by 1 then load, we can use $ldrbw1, [x0, 1]!$. The ! means increment after.
- $cbnzw1, length_rec$ checks if w1 is not zero. If it is not zero, then it branches to length_rec. This is the loop condition. If the character is not zero, then we continue the loop.
- $movx0, x2$ moves the length in x2 to x0. This is the return value.
- ret returns to the address in the link register.
- The rest is the same as before.

1.2 calling convention

Calling convention is not enforced by hardware, but it is a convention to make sure that the caller and callee agree on how to pass arguments and return values.

The caller is the function that calls another function. The callee is the function that is called. The calling convention is a contract between the caller and the callee. The caller must follow the calling convention to call

the callee. The callee must follow the calling convention to be called by the caller. The calling convention includes:

Here are some rules:

- bl is for address relative to the current instruction. b is for absolute address. blr is for register relative to the current instruction. br is for absolute register.
- function arguments are passed in registers x0-x7. If there are more than 8 arguments, then the rest are passed on the stack.
- function results are in x0. If there are more, we use x1 to x7 then the stack. x8 is used for system call number.
- return address is in lr (x30). lr is the link register. It is used to store the return address when we call a function. It is also used to store the address of the next instruction when we use bl.
- Caller is responsible for saving registers x0-x18. The callee can use them without saving them. The callee is responsible for saving registers x19-x30. The caller can use them without saving them.
- The stack grows downwards. The stack pointer is in x31. The stack pointer points to the last item in the stack. The stack pointer is decremented when we push something onto the stack. The stack pointer is incremented when we pop something from the stack.

1.3 call frame

The call frame is the part of the stack that is used to store information about the function call. It includes the return address, the saved registers, and the local variables.

The call frame is created when we call a function. It is destroyed when the function returns. The call frame is created by the caller. The caller is responsible for saving the registers and creating the local variables. The callee is responsible for saving the return address.

Usually we have stack pointer which points to the last item in the stack.

We also have frame pointer which points to the start of the call frame. The frame pointer is used to access the local variables.

The frame pointer is not necessary, but it makes it easier to access the local variables.

But if we have dynamically sized stack allocation (memory allocation functions), then fp is obligatory.

1.4 Jumping to a function

To jump to a function, we use bl. bl is a branch and link.

It branches to the function and saves the address of the next instruction in the link register. The function returns using ret, which branches to the address in the link register.

Back in the time when there is no bl, we have to do this manually. We have to save the return address in the stack, then jump to the function. The function then returns by popping the return address from the stack and jumping to it.

Another way is to specify the line of return when creating the callee function. And then when we call a function, the caller will write the return address to the specified line. And then the callee will jump to that line when it returns. This is called a trampoline. Not to nice to use, but it is a way to do it back in the day.

1.5 ABI

ABI is the application binary interface. It is a standard for binary interfaces. It specifies the calling convention, the system call numbers, and the system call arguments. It is used to make sure that the caller and callee agree on how to pass arguments and return values.

1.5.1 stack-based ABI

- stacks are sequential. If we spawn two functions, the stack must grow into two different directions. how to achieve this? We can use two different stacks.
- stack spaces are deallocated on return, so we can reuse the stack space.

2 Haxx

Start with a C example:

```c
int main() {
    volatile long ans = 42;
    char buf[16];
    gets(buf);
    //latex does not support printf properly
    printf(ans);
    return 0;
}
```

gets() reads a line from stdin into the buffer pointed to by s until either a terminating newline or EOF, which it replaces with a null byte ('\0'). No check for buffer overrun is performed (see BUGS below).

So this program is vulnerable to buffer overflow. We can overflow the buffer and overwrite the value of ans.

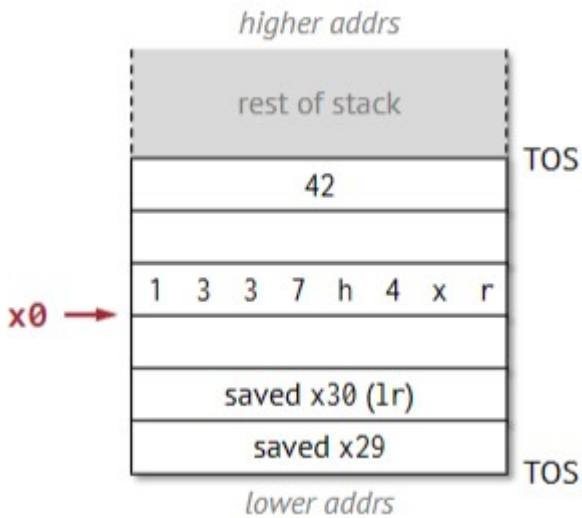
2.1 Analysis of the victim

By observing, volatile makes sure that ans lies on the stack. Then we allocated 16 bytes for buf (char is 1 byte each). Now the pointer of buf is 16 bytes away from ans. With that, have a look at the compiled assembly code:

```
main:
    stp x29, x30, [sp, -48]!
    mov x29, sp
    mov x0, 42
    str x0, [x29, 40]
    add x0, x29, 24
    bl gets
    ... rest of the code
```

stp is store pair. It stores two registers in the stack. The first register x29 (Stack pointer, pointing to the top of the stack of the parent function) is stored at the address in sp. The second register x30 (Link register, parent function address) is stored at the address in sp minus 8. The stack pointer is then decremented by 48. So 48 lines of memory is allocated for the call frame.

Then x29 keeps the current program sp. x0 is loaded with 42. Then x0 is stored in the address in x29 plus 40. So the address of ans is x29 plus 40 which is the bottom of the frame (Stack grows downwards, so right next to the parent function's top of stack). Then x0 is loaded with x29 plus 24. So x0 is loaded with the address of buf. Then we call gets.



Now if we input more than 16 bytes, we can overflow the buffer and overwrite the value of ans.

For example, we can input "ffffffff ffffffff 3905". This will overwrite the value of ans with 3905.

2.2 Typical attack pattern

- Find a program running on target (maybe as root)
- scan binary for potential buffer overflows. E.g, look for gets, strcpy, strcat, sprintf, etc.
- craft input that will overflow the buffer and overwrite something interesting
- objective: execute arbitrary code

2.2.1 Jump to arbitrary code

In ARMv8, typically, we save lr to the top of the stack before bl to another function.

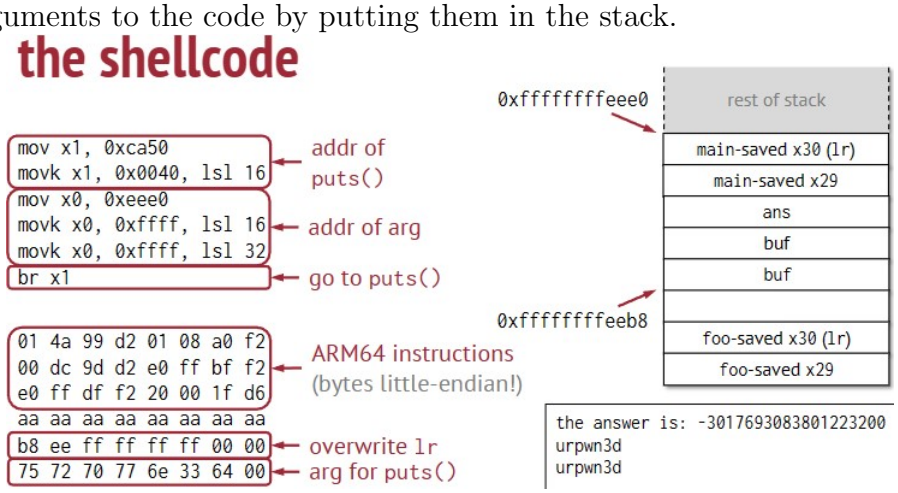
Since overwrite in this case writes to the higher address, but lr for the current call frame is at the lower address, we can't overwrite the current function's lr.

But we can overwrite the return address of the parent function which will lie just above the bottom of the call frame. So we can overwrite the return address of the parent function with the address of the code we want to execute. Then when the parent function returns, it will jump to the code we want to execute.

2.3 Code injection

Now we know how to jump to other addresses, but we still need to inject our code into the program. How do we pass arguments to the code you want to execute?

One way to do it is to overwrite the stack with the machine code that corresponds to the assembly code you want to execute. Then you can pass arguments to the code by putting them in the stack.



Note that the ARM64 code is little-endian. So the first byte is the least significant byte. We put the code to overwrite the space for buf and ans. The value printed is the third line of ARM machine code in large-endian.

we overwrite the lr with the start of the ARM code address "00 00 ff ff ff ff ee b8". So the pc will jump to your written code.

Also, since ARM only allows 16-bit constant input, we can load 16 bits of a register at a time. That is why we have to load the value of ans into x0 and x1 separately by shifting the constant input.

Now last line "br x1" would allow us to jump to the address in x1. The argument we want to pass has its address in x0, which is the last line we overwritten.

2.3.1 Code injection limitations

- stack/heap often non-executable.
  - enforced by virtual memory system (Segmentation fault)
  - It will crash the program if we try to execute code in the stack or heap.
- stack/heap locations often randomized (ASLR)
  - enforced by the virtual memory system
  - It will randomize the location of the stack and heap. So we can't predict the location of the stack or heap.

2.4 Gadgets

Gadgets are small pieces of code that end with a return instruction.

Since code injection is not executable, we can find some existing code (gadgets) and chain them together to do what we want. It could be either in the program itself or in a library.

2.4.1 return-oriented programming (ROP)

1. find gadgets in the program or libraries
2. put the address of the gadget in the stack by overflowing the buffer
3. prepare the arguments for the gadget by overflowing the buffer
4. construct a chain of gadgets that does what you want

Depending on the found gadgets, we inject the useful value into the stack by overflowing the buffer. The ultimate goal is to change the value of the link register and pass the desired arguments to the gadget.

Refer to mie's slides for examples.

2.4.2 more advanced ROP

- can be any indirect jump instruction, not just ret. For example

```
ldp x1, x2, [sp], 16
br x2
```

- can overwrite virtual function pointers in C++ objects.
- SROP: play with signals to set all registers.

2.5 Non-executable stack/data

For most systems, the stack and data are non-executable.

Virtual memory system enforces this. It will crash the program if we try to execute code in the stack or heap. Also called W^X policy (write xor execute).

However, we can use ROP to make a system call to make the stack executable. Then we can inject our code into the stack and execute it.

2.6 Address space layout randomization (ASLR)

ASLR randomizes the location of the stack and heap. So we can't predict the location of the stack or heap.

Virtual memory system enforces this. It will randomize the location of the stack and heap. So we can't predict the location of the stack or heap.

2.6.1 Limitations of ASLR

- not all bits are random, it is possible to brute-force the address space. Just need more trials.
- Library locations are not randomized since programs need to find libraries to use them once the program starts. So we can use gadgets in libraries.

- Programs on machines like servers are not restarted often. So we can use brute-force to find the address of the stack and heap.

2.7 Array bounds checking

Some programming languages like Java and Python have array bounds checking. It checks if the index is inside the bounds for each operation. So we can't overflow the buffer.

This prevents writing to memory that we don't own. However, for languages that could do pointer arithmetic, we can still do buffer overflow. C and C++ are typical examples.

Also, people sometimes disable array bounds checking for performance reasons. This kind of fast but unsafe code is hackers' best friend.

2.8 Stack canaries

- put some value on the stack below the return address (or frame pointer if it exists)
- check the value, see if it is modified before ret.
- if modified, abort the program. Might print something like "stack smashing detected" (Should work just like an exception)
- the variants can be 0, random, or random XOR with the return address.

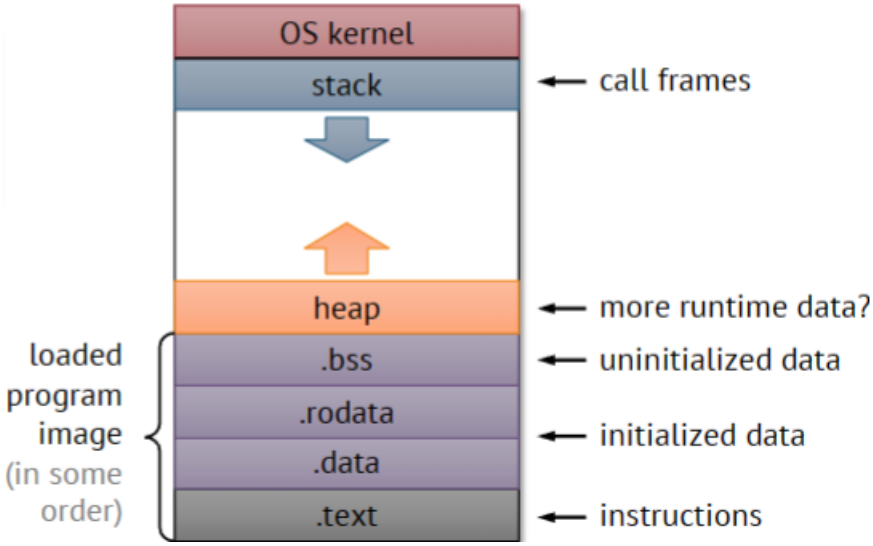
However, there are still limitations to it:

- if exception handling is hacked, we can still bypass the stack canaries.
- the pattern can only detect **some** overflows.
- fixed canary value can be easily discovered.
- canary value can still be leaked.
- canary feature is usually not enabled by default.

2.9 safe(r) programming languages

- Java, Python, etc. have array bounds checking.
- Rust, Go, etc. have memory safety.
- C/C++ are still the most popular language.
- safe languages are not always safe. For example, Java has a lot of vulnerabilities.
- Some languages guarantee safety via **type system**. For example, Rust. It is checked by the compiler at compile time. However, only guaranteed if used properly. Also, these languages are not as popular as C/C++.

3 Memory management



Each process has its own memory space. The memory space is divided into segments. The segments are:

- text segment: the code of the program
- data segment: the variables
- heap: the dynamic memory allocation

- stack: the local variables and the call frames

Heap and stack grow towards each other. The stack grows downwards. The heap grows upwards.

- Stack is easy to allocate but disappears after the function returns.
- Heap is harder to allocate but can be used after the function returns.

3.1 Allocation strategies

- automated:
  - garbage collection: the memory is automatically deallocated when it is no longer used. It is used in languages like Java and Python.
  - reference counting: the memory is automatically deallocated when the reference count is zero. It is used in languages like C++. (There are limitations to this, for example, circular references)
  - Hard to implement but easy to use
- manual: allocate and deallocate pairs.
  - malloc()+free() or new+delete comes in pairs. (It is a contract that must be obeyed)
  - easy to implement but hard to use. (lots of bugs)

3.2 Memory allocation

- want high utilization and low fragmentation
  - physical memory is limited among all processes
  - OS only gives big chunks of memory to processes (usually 4KB pages)
  - most allocs are small, few words to a few KB
- want high allocation throughput. Minimize the management overhead.

Recall the stack allocation:

- to allocate, just move the stack pointer downwards
- to deallocate, just move the stack pointer upwards
- easy to implement and efficient
- However it does not support dynamic allocation which does not work for heap.

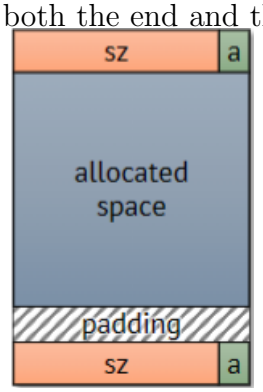
3.2.1 Challenges

- how to keep track of block size?
- how to keep track of free blocks?
- how to find a free block?
- how to allocate a block? which free block to use?

3.2.2 tracking allocation size

For each block, we can store the size of the block which can be used to find the start and the end of the current block. Thus we can find the neighboring blocks.

Note that the blocks can also request the size of both neighboring blocks. Just having the size at the start of the block is not enough. We need the size at both the end and the start of the block.



Note that we may or may not contain the padding there to follow the alignment rules of the system. At the same time, we can omit the lower bits of the size to store the status of the block since size should always occupy the whole line in the system no matter what the system is working with. Lastly,

the a there can indicate the status of the block. 1 means allocated, 0 means free.

3.2.3 finding a free block

let p = heap start: for each block b at p:

- if b is free **and** b is big enough, return b
- else $p = p$ + size of b
- keep going until the end of the heap
  - either we find a free block
  - or we run out of memory
  - or ask OS for more memory

This is a "first fit" strategy. It is simple and fast, but it can lead to fragmentation.

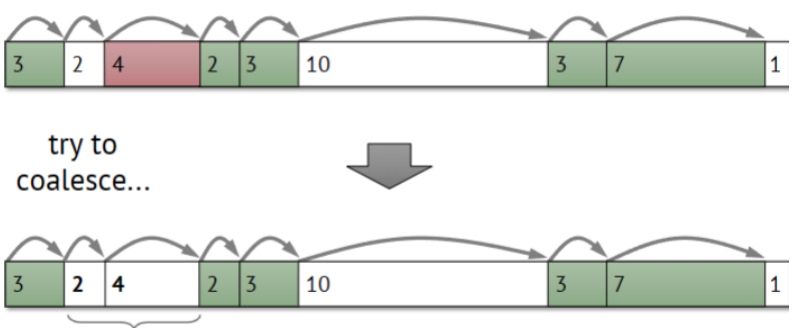
We can have "next fit" as well:

- keep track of the last block we allocated, return the next free and big enough block after that
- can be faster than first fit
- but still can lead to fragmentation (even worse than first fit)

Also we can have "best fit":

- return the smallest free block that is big enough
- but it is slow (overhead of searching for the smallest block)

3.2.4 coalescing and boundary tags



As shown in the figure, if we just free the 4-byte block, we will have two free blocks connecting together which we can merge by coalescing. This is done by checking the status of the neighboring blocks. If they are free, we can merge them together. This is called coalescing.

Note that we can use either the header or the boundary tag to track the neighboring blocks. So it does not matter if the free neighboring block is at the higher address or the lower address. We can always find it by checking the status and the size of the neighboring blocks.

In some cases, we might need to connect both neighboring blocks together if it happens that they are both free.